

Ventas-Backend

Glosario de estudio de la sesión

Cambios realizados el 2026-09-15 · Spring Boot 4 · Java 21 · Angular 21 · PostgreSQL

Explicaciones de razonamiento, líneas y bloques de código clave

Índice

1. Resumen del proyecto y stack
2. Seguridad y configuración: JWT_SECRET externalizado + application-local.yaml
3. JWT: filtro robusto ante tokens inválidos (500 → 401)
4. Borrado de productos con histórico: del 500 al 409
5. Categorías y roles: fin de la dependencia de lazy-loading
6. Imágenes huérfanas: 404 silencioso (NoResourceFoundException)
7. Permisos: limpieza de huérfanos (38→34) + asimetría CAJERO
8. Stock 0 permitido (frontend)
9. Glosario de conceptos usados
10. Comandos para verificar

1. Resumen del proyecto y stack

Ventas-Backend es un ERP/POS (punto de venta) que el usuario usa en una tienda. Arquitectura en capas: **controller** → **service (interfaz)** → **service/impl** → **repository** → **model/entity**. DTOs en model/dto, Mappers en mapper, excepciones en exceptions.

- **Spring Boot 4.0.5** · Java 21 · Maven (mvnw.cmd)
- **PostgreSQL** ventas_db con Spring Data JPA (ddl-auto: update)
- **Spring Security + JWT** (jjwt 0.11.5, HS256, expiración 5h)
- **13 controllers, 54 endpoints**, 34 permisos, autorización por @PreAuthorize
- **Terminal de pagos** abstracta (SIMULATED o PHYSICAL por socket)
- **Frontend Angular 21** en carpeta hermana Ventas-Frontend (consumo /api)
- 4 **CommandLineRunner** de bootstrap: Role → Admin → Permission → RolePermission

El objetivo de la sesión fue: **a) preparar el proyecto para correr en local y en Railway** externalizando secretos, **b) corregir errores que el usuario encontró al probar la app** (login roto, borrado de producto con 500, campo stock que no aceptaba 0) y **c) documentar** el código con comentarios didácticos para aprendizaje futuro.

2. Seguridad y configuración: JWT_SECRET externalizado + application-local.yaml

El problema

Antes, el secreto del JWT estaba **hardcodeado en el código**. Eso es una mala práctica de seguridad (si publicas el repo, cualquiera puede firmar tokens válidos). En la sesión anterior se reescribió application.yaml para usar variables de entorno \${VAR} sin defaults, y se generó un JWT_SECRET nuevo.

application-local.yaml (src/main/resources, gitignored)

```
# SOLO DESARROLLO LOCAL (NUNCA subir a git)
DB_USER: postgres
DB_PASSWORD: admin
MAIL_USERNAME: soportetiav@gmail.com
MAIL_PASSWORD: "tucn dfbu qspk zzki" # App password de Gmail (NO la contraseña normal)
JWT_SECRET: ZGV2LXZlbnRhcylqd3QtY2Q0ZDZlNjUtZTc5OS00ODEyLWI0YjctOTAzMjY4NDY0MGNhLXNlY3JldC1rZXktMjAyNi0=
ADMIN_EMAIL: 18jarquinsanchezerik1a@gmail.com
ADMIN_PASSWORD: "1234"
PAYMENT_MERCHANT_ID: "MERCHANT12345"
PAYMENT_TERMINAL_ID: "TERMINAL001"
PAYMENT_KEYSTORE_PASSWORD: changeit
CORS_ALLOWED_ORIGINS: "http://localhost:4200"
```

Por qué importa

- **Las compañías de correo ya no usan la contraseña normal:** Gmail exige una “Contraseña de aplicación” generada desde Verificación en 2 pasos. El valor con espacios va entre comillas en YAML.
- **El JWT_SECRET debe tener al menos 32 bytes.** Se genera con openssl rand -base64 48. Es sensible: si cambia, todos los tokens viejos dejan de validarse (los usuarios deben re-loguearse).
- En **Railway** este archivo **no existe**: los \${VAR} se resuelven desde las Variables del servicio. Por eso el archivo está en .gitignore.

Lección clave (efecto colateral que vimos)

Al cambiar el JWT_SECRET, el token que el navegador guardó en **localStorage** quedó firmado con el secreto viejo. El **interceptor de Angular** agrega ese token a TODAS las peticiones, incluso el login. Como el filtro solo cazaba ExpiredJwtException, la SignatureException (firma no coincide) explotaba con 500 y el login fallaba. De ahí el cambio del punto 3.

3. JWT: filtro robusto ante tokens inválidos (500 → 401)

Archivos

- src/main/java/com/erikjarquin/ventas/config/JwtFilter.java

El error que vimos

```
io.jsonwebtoken.security.SignatureException:  
JWT signature does not match locally computed signature.
```

Ocurría en JwtUtil.getClaims → extractEmail → JwtFilter.doFilterInternal. Como el catch solo contemplaba ExpiredJwtException, cualquier otra excepción de JWT se escapaba al manejador global y respondía 500.

El fix (líneas 88-95)

```
} catch (JwtException | IllegalArgumentException e) {  
    // Token inválido: expirado, malformado o firmado con otro secreto  
    // (SignatureException). No lanzamos 500: limpiamos contexto y dejamos  
    // seguir la cadena sin autenticar → los endpoints protegidos dan 401.  
    SecurityContextHolder.clearContext();  
    filterChain.doFilter(request, response);  
    return;  
}
```

Razonamiento

JwtException es la clase PADRE de ExpiredJwtException y SignatureException; capturándola cubrimos token expirado, malformado y mal firmado a la vez. Un token inválido **no debe tumbar el request**: se limpia el contexto (dejar sin autenticación) y la cadena continúa. El endpoint público (login) funciona; los protegidos devuelven 401/403.

Nota didáctica sobre JWT en este proyecto

- El JWT **solo guarda el email del usuario** (subject). Los permisos se cargan **siempre de la BD** en cada request (findByEmailWithRoleAndPermissions), así que al quitar un permiso a un rol, el cambio aplica de inmediato sin esperar la expiración del token.
- **Header: Authorization: Bearer <token>**. El filtro extrae con substring(7) la parte después de “Bearer”.
- **Se crea un UsernamePasswordAuthenticationToken con las autoridades (ROLE_* + permisos) y se deposita en SecurityContextHolder; para eso inyectamos SecurityAuthorityMapper.**

4. Borrado de productos con histórico: del 500 al 409

Archivos

- service/impl/ProductImpl.java (método delete)
- repository/ProductRepository.java (2 métodos nuevos)
- exceptions/ProductException.java (NUEVO)
- exceptions/GlobalExceptionHandler.java (handler PRODUCT_ERROR)

El error que vimos

```
ERROR: update or delete on table "products" violates foreign key constraint
"fkqvh82ispfukxa2yssnok0m12o" on table "sale_details"
Detail: Key (id)=(3) is still referenced from table "sale_details".
```

El método original simplemente hacía `repository.deleteById(id)`. Cuando el producto ya había sido vendido (o comprado), la BD rechazaba el DELETE por la llave foránea y el usuario recibía un 500 genérico sin explicación.

El fix: validar histórico ANTES de borrar

```
//Borrar producto
@Override
public void delete(Long id){
    ProductEntity entity = repository.findById(id).orElseThrow(
        () -> new ProductException("Producto no encontrado"));

    //No se puede borrar un producto con histórico de ventas o compras:
    //la venta/compra lo referencia por FK y perderíamos el dato histórico.
    if(repository.existsBySaleDetailsProductId(id)
        || repository.existsByPurchaseDetailsProductId(id)){
        throw new ProductException(
            "No se puede eliminar el producto: tiene ventas o compras asociadas. "
            + "Puedes editar su stock en 0 o dejar de usarlo en el punto de venta",
            HttpStatus.CONFLICT);
    }

    fileStorageService.delete(entity.getImg());
    repository.deleteById(id);
}
```

Los métodos nuevos en ProductRepository (JPQL sobre la entidad detalle)

```
//¿El producto se ha vendido alguna vez? (para bloquear su borrado con 409)
@Query("SELECT COUNT(d) > 0 FROM SaleDetailEntity d WHERE d.product.id = :productId")
boolean existsBySaleDetailsProductId(@Param("productId") Long productId);

//¿El producto se ha comprado alguna vez? (para bloquear su borrado con 409)
@Query("SELECT COUNT(d) > 0 FROM PurchaseDetailEntity d WHERE d.product.id = :productId")
boolean existsByPurchaseDetailsProductId(@Param("productId") Long productId);
```

Razonamiento

Es un **conflicto de negocio**: el borrado físico rompería el histórico (los renglones de venta apuntan al producto por FK). La respuesta adecuada es **409 Conflict** con un mensaje claro, igual que ya hacían categorías (con productos) y roles (con usuarios). El borrado físico solo es válido para productos que NUNCA se vendieron ni compraron; de resto, la alternativa es dejarlos con stock 0.

ProductException.java (patrón del proyecto)

```
public class ProductException extends RuntimeException {
    private final HttpStatus status;
    public ProductException(String message) {
        this(message, HttpStatus.NOT_FOUND);          // por defecto 404
    }
    public ProductException(String message, HttpStatus status) {
        super(message);
        this.status = status;
    }
    public HttpStatus getStatus() { return status; }
}
```

Handler en GlobalExceptionHandler

```
@ExceptionHandler(ProductException.class)
public ResponseEntity<ErrorResponse> handleProductException(ProductException ex) {
    log.warn("Error de producto: {}", ex.getMessage());
    return buildErrorResponse(ex.getStatus(), "PRODUCT_ERROR", ex.getMessage());
}
```

5. Categorías y roles: fin de la dependencia de lazy-loading

Archivos

- service/impl/CategoryImpl.java (delete)
- service/impl/RoleImpl.java (deleteRole)
- repository/ProductRepository.java (countByCategory_Id)
- repository/UserRepository.java (countByRole_Id)

El problema (código original)

```
//ANTES - CategoryImpl.delete
CategoryEntity category = repository.findById(id).orElseThrow(...);
if(category.getProducts() != null && !category.getProducts().isEmpty()){ // ■■ Lazy fuera de transacción
    throw new CategoryException("...", HttpStatus.CONFLICT);
}

//ANTES - RoleImpl.deleteRole
RoleEntity role = repository.findById(id).orElseThrow(...);
if(role.getUsers() != null && !role.getUsers().isEmpty()){ // ■■ Lazy fuera de transacción
    throw new RoleException("...", HttpStatus.CONFLICT);
}
```

`getProducts()` y `getUsers()` son colecciones **LAZY** (se cargan de la BD solo cuando se acceden). El método `delete` no es `@Transactional`, así que al salir de `findById` la sesión de Hibernate puede estar cerrada → `LazyInitializationException` o depende del open-in-view. Frágil.

El fix: conteos SQL derivados

```
//ProductRepository
long countByCategory_Id(Long categoryId);

//UserRepository
long countByRole_Id(Long roleId);

//AHORA - CategoryImpl.delete
repository.findById(id).orElseThrow(() -> new CategoryException("Categoría no encontrada"));
if(productRepository.countByCategory_Id(id) > 0){
    throw new CategoryException("No puedes eliminar una categoría con productos", HttpStatus.CONFLICT);
}
repository.deleteById(id);
```

Razonamiento

Un **conteo SQL** (`countBy...`) hace que la BD responda un número y no dependemos de cargar colecciones. Mismo resultado de negocio (409), pero sin sorpresas de lazy loading ni de transacción. Nota del naming: `countByCategory_Id` usa guion bajo para separar el atributo anidado (`category.id`) del nombre del método.

6. Imágenes huérfanas: 404 silencioso (NoResourceFoundException)

El problema

El bug original de producto borraba la imagen de disco ANTES de que el DELETE fallara por FK. Resultado: productos que siguen en BD pero cuya img apunta a un archivo inexistente. Al cargar la tabla, el navegador pedía /api/uploads/x.jpg y Spring lanzaba NoResourceFoundException → 500 (spam de logs y pantallas rotas).

El fix en GlobalExceptionHandler

```
@ExceptionHandler({NoResourceFoundException.class})
public ResponseEntity<ErrorResponse> handleNoResource(NoResourceFoundException ex) {
    return buildErrorResponse(HttpStatus.NOT_FOUND, "NOT_FOUND",
        "El recurso solicitado no existe");
}
```

Razonamiento

Una imagen que ya no existe es un **404 normal**, no un error del servidor. Con este handler, el frontend simplemente no muestra la imagen (broken image) y la app sigue trabajando. Otra lección: **no borres el archivo físico hasta estar seguro de que el cambio en BD va a persistir.**

7. Permisos: limpieza de huérfanos (38→34) + asimetría CAJERO

Limpieza de permisos sin uso

Se revisó el enum `PermissionName` y se eliminaron 4 permisos que ningún endpoint ni guard de frontend usaban:

- `CANCELAR_VENTAS` (no existía la operación)
- `EXPORTAR_REPORTES` (la exportación se hace 100% en el frontend)
- `VER_CONFIGURACION` y `EDITAR_CONFIGURACION` (módulo inexistente)
- Se conservaron `VER_CLIENTES` y `VER_FACTURAS`: el frontend los usa como guard en las rutas `/clientes` y `/facturas`.

El enum quedó con javadoc que explica cómo sincronizar cambios: `PermissionBootstrap` crea los permisos en BD al arrancar y `RolePermissionBootstrap` los reasigna a cada rol (self-healing).

Asimetría CAJERO corregida

El rol CAJERO podía abrir/cerrar caja pero NO consultar su estado (`GET /api/cash/active`) → al cargar el POS el frontend recibía 403. Se agregó `VER_CAJA` en `RolePermissionBootstrap`:

```
cajero.setPermissions(filterPermissions(  
    allPermissions,  
    PermissionName.VER_PRODUCTOS,  
    PermissionName.VER_VENTAS,  
    PermissionName.CREAR_VENTAS,  
    PermissionName.VER_CAJA,           // <-- necesario para leer si hay caja abierta  
    PermissionName.ABRIR_CAJA,  
    PermissionName.CERRAR_CAJA  
));
```

Nota local: Hibernate genera un `CHECK` constraint para `@Enumerated(String)` que `ddl-auto:update` NO actualiza. Al agregar permisos al enum, en BD local el arranque puede fallar con “viola la restricción check” → soltar el constraint a mano o usar BD nueva.

8. Stock 0 permitido (frontend)

Archivos (Ventas-Frontend)

- src/app/features/products/productos.html (campo stock)
- src/app/features/products/productos.ts (validateNumber)

El cambio

```
<!-- ANTES -->
<input [(ngModel)]="form.stock" min="1" (input)="validateNumber('stock')" type="number"/>

<!-- AHORA -->
<input [(ngModel)]="form.stock" min="0" (input)="validateNumber('stock')" type="number"/>

//ANTES
validateNumber(field: 'price' | 'stock'){
  if(this.form[field] < 1){ this.form[field] = 1; }
}

//AHORA
validateNumber(field: 'price' | 'stock'){
  // El stock puede ser 0 (agotado); el precio no puede bajar de 1.
  const min = field === 'stock' ? 0 : 1;
  if(this.form[field] < min){ this.form[field] = min; }
}
```

Razonamiento

Stock 0 es un estado válido (producto agotado). En el backend ProductImpl acepta el entero tal cual (no valida mínimo), así que el bloqueo estaba SOLO en el frontend. Importante: **stock 0 NO habilita el borrado** — el borrado se bloquea por histórico de ventas/compras, no por el stock.

9. Glosario de conceptos usados

@PreAuthorize("hasAuthority('...')")

Anotación de Spring Security que verifica PERMISOS antes de invocar el endpoint. El string debe coincidir EXACTAMENTE con un valor de PermissionName (el mismo que se guarda en BD). Si el usuario autenticado no tiene esa autoridad → 403.

Filtro JWT (OncePerRequestFilter)

Clase que se ejecuta en CADA request antes que el controller. Lee el header Authorization, valida el token y arma la autenticación en el SecurityContext. Sin un filtro así, Spring Security no sabría quién es el usuario.

SecurityContextHolder

Almacén (thread-local) donde Spring Security guarda la autenticación del request actual. Al limpiarlo con clearContext() dejamos la petición "sin usuario".

JwtException / ExpiredJwtException / SignatureException

Jerarquía de jwt: SignatureException significa que la firma no coincide con el secreto actual; ExpiredJwtException que pasó la expiración (5h). Capturar la clase padre cubre todos los casos.

DataIntegrityViolationException / FK

Error de BD cuando un DELETE/UPDATE choca con una llave foránea. Es señal de que intentas borrar un registro que está referenciado. En este proyecto se convierte en 409 con mensaje claro, no en 500.

HttpStatus.CONFLICT (409)

Respuesta para conflictos de estado: nombre duplicado, registro referenciado por otros, etc. Contrasta con 400 (entrada inválida), 404 (no encontrado), 403 (sin permiso).

404 vs 500

404 = el recurso no existe (cliente pidió algo que no está). 500 = el servidor falló. Recursos estáticos inexistentes (imágenes) deben ser 404, no 500.

Lazy vs Eager loading

Lazy: la colección se carga SOLO al accederla (requiere sesión abierta). Eager: se carga junto con la entidad. Fuera de una transacción, tocar una colección lazy lanza LazyInitializationException.

open-in-view

Opción de JPA que mantiene la sesión abierta durante toda la petición web (cubre los accesos lazy fuera de @Transactional). Conveniente en desarrollo, pero se recomienda no depender de ella.

Queries derivadas vs @Query

Spring Data genera SQL a partir del NOMBRE del método (findByName, countByRole_Id), o recibe JPQL con @Query. El estilo derivado es más simple; @Query da control total (JOINS, COUNT, agrupaciones).

@EntityGraph

Estrategia de carga para precargar relaciones en UNA consulta y evitar N+1 (usado en PurchaseRepository: provider + details.product).

Bootstrap / CommandLineRunner

Código que se ejecuta al arrancar la app (@Order controla el orden). Se usan para crear roles, admin, permisos y su asignación (seed). RolePermissionBootstrap es self-healing: reasigna en cada arranque.

CommandLineRunner + @Transactional

Los bootstraps deben ser @Transactional para que múltiples operaciones de BD dentro de run() formen una sola transacción coherente.

YAML con valores entre comillas

Si un valor tiene espacios o caracteres especiales (como la app password de Gmail), debe ir entre comillas dobles para que el parser YAML no lo rompa.

@RestControllerAdvice

Manejador global de excepciones: centraliza la conversión de errores en un JSON uniforme { timestamp, status, code, message }. Cada excepción de negocio tiene su @ExceptionHandler.

MultipartFile / FormData

El CRUD de productos recibe la imagen como archivo multipart y el frontend la envía con FormData.

FileStorageService valida tamaño (5MB), extensión y magic bytes antes de guardar en disco.

CORS

El frontend corre en http://localhost:4200 y el backend en :8081: son orígenes distintos. SecurityConfig configura CORS global con CORS_ALLOWED_ORIGINS (split por comas).

10. Comandos para verificar

```
.\mvnw.cmd compile          # compilar backend
.\mvnw.cmd test              # 35 tests (requiere PostgreSQL local levantado)
.\mvnw.cmd spring-boot:run   # levantar backend en :8081

cd ../Ventas-Frontend
npx ng build                  # compilar frontend
npx ng serve                  # dev server :4200

# Generar un JWT_SECRET fuerte:
openssl rand -base64 48
```

Nota: antes de publicar el repo, purgar el historial de git (git filter-repo) porque contiene secretos viejos. application-local.yaml ya está en .gitignore.